MPT Circuit (Merkle Patricia Trie Circuit)

목적

MPT Circuit은 **이더리움 상태 트리의 업데이트를 검증**하는 회로입니다. Scroll은 zkTrie (Sparse Binary Merkle Patricia Trie)를 사용하여 Poseidon 해시 기반의 효율적인 영지식 증명을 제공합니다. 이 회로는 계정 상태 변경과 스토리지 업데이트에 대한 Merkle 증명을 생성하고 검증합니다.

zkTrie 구조

Sparse Binary Merkle Patricia Trie

Scroll은 이더리움의 전통적인 Hexary MPT 대신 **zkTrie**를 사용합니다. 이는 SNARK-friendly한 구조로 설계되었습니다.

파일: `zktrie/src/state.rs`

주요 특징

1. **Binary Tree**: 각 노드는 최대 2개의 자식 (left, right)
2. **Sparse**: 대부분의 노드가 비어있음 (효율적인 저장)
3. **Poseidon Hash**: SNARK-friendly 해시 함수 사용
4. **Path Compression**: 긴 경로를 압축하여 저장

트리 구조

```
Root (Poseidon Hash)

/ \
```

```
/ \

Node (left) Node (right)

/ \ / \

Leaf1 Leaf2 Leaf3 Leaf4

(Account) (Account) (Storage) (Storage)
```

노드 타입:

- **Leaf Node** (Type 4): 실제 데이터를 저장 (계정 또는 스토리지)
- **Middle Node** (Type 6-9): 중간 노드, 자식 노드의 해시를 저장
- **Empty Node** (Type 5): 빈 노드

파일: `zktrie/src/state/builder.rs` (line 30-35)

```rust
pub(crate) const NODE_TYPE_MIDDLE_0: u8 = 6;

pub(crate) const NODE_TYPE_MIDDLE_1: u8 = 7;

pub(crate) const NODE_TYPE_MIDDLE_2: u8 = 8;

pub(crate) const NODE_TYPE_MIDDLE_3: u8 = 9;

pub(crate) const NODE_TYPE_LEAF: u8 = 4;

pub(crate) const NODE_TYPE_EMPTY: u8 = 5;
```

zkTrie vs 이더리움 MPT

| 특징 | 이더리움 MPT | zkTrie (Scroll) |

|------|---------|---------|

| **해시 함수** | Keccak256 | Poseidon |

| **트리 구조** | Hexary (16개 자식) | Binary (2개 자식) |

| **경로 길이** | 64 nibbles (256 bits) | 256 bits |

| **회로 효율성** | 낮음 (Keccak 비용 높음) | 높음 (Poseidon SNARK-friendly) |

| **증명 크기** | 큼 | 작음 |

AccountData 구조

파일: `zktrie/src/state/builder.rs` (line 38-49)

```
pub struct AccountData {
pub nonce: u64, // 계정 nonce
pub balance: U256, // 계정 잔액
pub keccak_code_hash: H256, // Keccak 코드 해시
pub poseidon_code_hash: H256, // Poseidon 코드 해시
pub code_size: u64, // 코드 크기
pub storage_root: H256, // 스토리지 트리 루트
}
```

필드 인코딩:

- Field 0: `[code_size (8 bytes) | nonce (8 bytes)]` (16 bytes, padded to 32)
- Field 1: `balance` (32 bytes)
- Field 2: `storage_root` (32 bytes)
- Field 3: `keccak_code_hash` (32 bytes)
- Field 4: `poseidon_code_hash` (32 bytes)

StorageData 구조

파일: `zktrie/src/state/builder.rs` (line 127-136)

```
pub struct StorageData(Word); // 32바이트 스토리지 값
```

스토리지는 단순히 32바이트 값으로 저장됩니다.

SMTTrace와 Proof 변환

SMTTrace 구조

파일: `zkevm-circuits/src/witness/mpt/witness.rs` (line 1-20)

MPT Circuit은 `SMTTrace` 구조를 사용하여 상태 업데이트를 표현합니다:

```
pub struct SMTTrace {
pub address: HexBytes, // 계정 주소
pub account_path: [SMTPath; 2], // [before, after] 계정 경로
pub account_update: [Option<SMTAccount>; 2], // [before, after] 계정 데이터
pub account_key: SMTHash, // 계정 키 (해시된 주소)
pub state_path: [Option<SMTPath>; 2], // [before, after] 스토리지 경로
pub common_state_root: Option<SMTHash>, // 공통 스토리지 루트
pub state_key: Option<SMTHash>, // 스토리지 키 (해시된 키)
pub state_update: Option<[Option<StateData>; 2]>, // [before, after] 스토리지 데
}
```

SMTPath 구조

```rust
pub struct SMTPath {
pub root: SMTHash, // 트리 루트 해시
pub leaf: Option<SMTNode>, // 리프 노드
pub path: Vec<SMTNode>, // 루트에서 리프까지의 경로
pub path_part: BigUint, // 경로 비트맵 (0=left, 1=right)
}
```

SMTNode 구조

```rust
pub struct SMTNode {
pub node_type: u64, // 노드 타입 (4-9)
pub value: SMTHash, // 노드 값 (현재 노드 해시)
pub sibling: SMTHash, // 형제 노드 해시
}
```

Proof 변환

파일: `zkevm-circuits/src/mpt_circuit.rs` (line 100-110)

```rust
let traces: Vec<_> = block
.mpt_updates
.proof_types
.iter()
.cloned()
```

```
.zip_eq(block.mpt_updates.smt_traces.iter().cloned())
.collect();


Self {
proofs: traces.into_iter().map(Proof::from).collect(),
// ...
}
```

`Proof` 구조는 `(MPTProofType, SMTTrace)` 튜플에서 생성됩니다.

PathType (경로 타입)

MPT Circuit은 경로를 세 가지 타입으로 분류합니다:

1. Common (공통 경로)

- **의미**: 업데이트 전후에 공통으로 존재하는 경로
- **사용**: 대부분의 노드가 변경되지 않은 경우
- **예시**: 스토리지 업데이트 시 계정 트리의 대부분 경로

```
Before: Root → A → B → C → Leaf (old value)
After: Root → A → B → C → Leaf (new value)
Common: Root → A → B → C (변경되지 않음)
```

2. ExtensionNew (새 확장 경로)

- **의미**: 업데이트 후에만 존재하는 경로
- **사용**: 새 노드가 추가된 경우
- **예시**: 새 계정 생성, 새 스토리지 슬롯 생성

```
Before: Root → A → Empty

After: Root → A → B → C → Leaf (new)

ExtensionNew: B → C → Leaf
```

3. ExtensionOld (이전 확장 경로)

- **의미**: 업데이트 전에만 존재하는 경로
- **사용**: 노드가 삭제된 경우
- **예시**: 계정 삭제, 스토리지 슬롯 삭제

```
Before: Root → A → B → C → Leaf (old)

After: Root → A → Empty

ExtensionOld: B → C → Leaf
```

SegmentType (세그먼트 타입)

MPT Circuit은 증명을 여러 세그먼트로 나눕니다. 각 세그먼트는 특정 타입의 데이터를 처리합니다.

계정 트리 세그먼트

1. AccountTrie

- **의미**: 계정 트리의 경로

- **데이터**: 계정 키에서 계정 리프까지의 경로
- **길이**: 가변 (경로 깊이에 따라)

2. AccountLeaf0-4

계정 리프는 5개의 필드로 구성되며, 각 필드는 별도의 세그먼트로 처리됩니다:

- **AccountLeaf0**: `[code_size | nonce]` (16 bytes)
- **AccountLeaf1**: `balance` (32 bytes)
- **AccountLeaf2**: `storage_root` (32 bytes)
- **AccountLeaf3**: `keccak_code_hash` (32 bytes)
- **AccountLeaf4**: `poseidon_code_hash` (32 bytes)

스토리지 트리 세그먼트

3. StorageTrie

- **의미**: 스토리지 트리의 경로
- **데이터**: 스토리지 키에서 스토리지 리프까지의 경로
- **길이**: 가변 (경로 깊이에 따라)

4. StorageLeaf0-1

스토리지 리프는 2개의 세그먼트로 처리됩니다:

- **StorageLeaf0**: 스토리지 값의 상위 16 bytes
- **StorageLeaf1**: 스토리지 값의 하위 16 bytes

세그먼트 처리 순서

```
1. AccountTrie (경로 검증)
```

```
2. AccountLeaf0-4 (계정 필드 검증)

3. StorageTrie (스토리지 경로 검증, 필요 시)

4. StorageLeaf0-1 (스토리지 값 검증, 필요 시)
```

비존재 증명 (Non-Existence Proof)

zkTrie는 두 가지 타입의 비존재 증명을 지원합니다.

Type 1: Insert to/Delete from Append

의미: 트리의 끝에 노드를 추가하거나 삭제

시나리오:

- 새 계정 생성 (트리에 계정이 없음)
- 새 스토리지 슬롯 생성 (스토리지 키가 없음)

증명 구조:

```
Before: Root → A → B → Empty

After: Root → A → B → Leaf (new)
```

검증:

1. Before 경로가 Empty 노드로 끝남
2. After 경로가 새 Leaf 노드로 끝남
3. 경로의 나머지 부분은 동일

파일: `zkevm-circuits/src/witness/mpt/witness.rs` (line 450-470)

```
MPTProofType::AccountDoesNotExist => {
```

```
// 계정이 존재하지 않음을 증명

assert!(acc_before.is_none());

assert!(acc_data.balance.is_zero());

assert_eq!(0, acc_data.nonce);

assert!(acc_data.storage_root.is_zero());

return None;

}
```

Type 2: Insert to/Delete from Fill

의미: 트리의 중간에 노드를 추가하거나 삭제 (기존 Empty 노드를 채움)

시나리오:

- 이전에 삭제된 계정 위치에 새 계정 생성
- 이전에 삭제된 스토리지 슬롯에 새 값 저장

증명 구조:

```
Before: Root → A → Empty → (sibling exists)

After: Root → A → Leaf (new) → (sibling exists)
```

검증:

1. Before 경로가 Empty 노드로 끝남
2. 형제 노드(sibling)가 존재함 (Empty가 아님)
3. After 경로가 새 Leaf 노드로 끝남
4. 형제 노드는 변경되지 않음

Type 1과 Type 2의 차이:

- **Type 1**: 형제 노드가 없음 (트리의 끝)
- **Type 2**: 형제 노드가 존재함 (트리의 중간)

비존재 증명 예시

새 계정 생성 (Type 1)

```
// Before: 계정이 존재하지 않음
SMTTrace {
account_path: [
SMTPath { root, leaf: None, path: [...], path_part }, // Empty로 끝남
SMTPath { root, leaf: Some(new_leaf), path: [...], path_part }
],
account_update: [None, Some(new_account)],
// ...
}
```

삭제된 계정 위치에 생성 (Type 2)

```
// Before: Empty 노드이지만 형제 노드 존재
SMTTrace {
account_path: [
SMTPath {
root,
```

```
leaf: None, // Empty

path: [..., SMTNode { sibling: non_empty_hash }], // 형제 존재

path_part

},

SMTPath { root, leaf: Some(new_leaf), path: [...], path_part }

],

account_update: [None, Some(new_account)],

// ...

}
```

MPTProofType별 제약 조건

계정 필드 업데이트

1. NonceChanged (Nonce 변경)

제약 조건:

```
// old_val은 u64 범위

assert!(old_val <= u64::MAX.into());

// new_val은 u64 범위 (CREATE 시 1 << 64 허용)

assert!(new_val <= Word::from(u64::MAX) + Word::one());

// old_val이 현재 nonce와 일치

assert_eq!(old_val.as_u64(), acc_data.nonce);
```

업데이트:

```
acc_data.nonce = new_val.as_u64();
```

사용 예시:

- 트랜잭션 실행 시 발신자 nonce 증가
- 컨트랙트 생성 시 nonce 초기화

2. BalanceChanged (잔액 변경)

제약 조건:

```
// old_val이 현재 balance와 일치
assert_eq!(old_val, acc_data.balance);
```

업데이트:

```
acc_data.balance = new_val;
```

사용 예시:

- ETH 전송
- 가스 지불
- 컨트랙트 실행 보상

3. CodeHashExists (Keccak 코드 해시)

제약 조건:

```
// old_val이 현재 keccak_code_hash와 일치
let mut code_hash = [0u8; 32];
```

```
old_val.to_big_endian(code_hash.as_mut_slice());

debug_assert_eq!(H256::from(code_hash), acc_data.keccak_code_hash);
```

업데이트:

```
new_val.to_big_endian(code_hash.as_mut_slice());

acc_data.keccak_code_hash = H256::from(code_hash);
```

사용 예시:

- 컨트랙트 배포 시 코드 해시 설정
- EXTCODEHASH opcode

4. PoseidonCodeHashExists (Poseidon 코드 해시)

제약 조건:

```
// old_val이 현재 poseidon_code_hash와 일치

let mut code_hash = [0u8; 32];

old_val.to_big_endian(code_hash.as_mut_slice());

debug_assert_eq!(H256::from(code_hash), acc_data.poseidon_code_hash);
```

업데이트:

```
new_val.to_big_endian(code_hash.as_mut_slice());

acc_data.poseidon_code_hash = H256::from(code_hash);
```

사용 예시:

- zkTrie에서 코드 검증
- Poseidon 해시 기반 증명

5. CodeSizeExists (코드 크기)

제약 조건:

```
// old_val과 new_val은 u64 범위
assert!(old_val < u64::MAX.into());
assert!(new_val < u64::MAX.into());
// old_val이 현재 code_size와 일치
debug_assert_eq!(old_val.as_u64(), acc_data.code_size);
// 코드 크기는 0에서만 변경 가능
if !(old_val.as_u64() == 0u64 || old_val.as_u64() == new_val.as_u64()) {
log::warn!("MPTProofType::CodeSizeExists {old_val}=>{new_val}");
}
```

업데이트:

```
acc_data.code_size = new_val.as_u64();
```

사용 예시:

- EXTCODESIZE opcode
- 컨트랙트 배포 시 코드 크기 설정

6. AccountDoesNotExist (계정 비존재)

제약 조건:

```
// 계정이 존재하지 않아야 함
assert!(acc_before.is_none());
assert!(acc_data.balance.is_zero());
assert_eq!(0, acc_data.nonce);
assert!(acc_data.storage_root.is_zero());
```

업데이트:

```
return None; // 계정을 생성하지 않음
```

사용 예시:

- 새 계정 생성 전 비존재 확인
- CREATE/CREATE2 opcode

스토리지 업데이트

7. StorageChanged (스토리지 변경)

제약 조건:

```
// old_value가 트리의 현재 값과 일치
let  old_value_in_trie = storage_before
.as_ref()
.ok()
.and_then(|(_, node)| node.as_ref())
.and_then(|node| node.as_storage())
```

```
.unwrap_or_default();

assert_eq!(hex::encode(old_value), hex::encode(old_value_in_trie));
```

업데이트:

```
if !new_value.is_zero() {

trie.update_store(key.as_ref(), &new_value).unwrap();

} else  if !old_value.is_zero() {

trie.delete(key.as_ref());

}
```

사용 예시:

- SSTORE opcode
- 스토리지 값 변경

특수 케이스:

- `new_value == 0 && old_value == 0`: 트리를 건드리지 않음
- `new_value != 0`: 값 업데이트
- `new_value == 0 && old_value != 0`: 키 삭제

8. StorageDoesNotExist (스토리지 비존재)

제약 조건:

```
// 스토리지 키가 존재하지 않아야 함

assert!(old_value.is_zero());

assert!(new_value.is_zero());
```

사용 예시:

- 새 스토리지 슬롯 생성 전 비존재 확인
- SLOAD opcode (존재하지 않는 키)

계정 삭제

9. AccountDestructed (계정 삭제)

제약 조건:

```
// 계정이 존재해야 함
assert!(acc_before.is_some());
// 삭제 후 계정이 없어야 함
assert!(acc_after.is_none());
```

업데이트:

```
trie.delete(address.as_bytes());
```

사용 예시:

- SELFDESTRUCT opcode
- 계정 완전 삭제

회로 구조

MptCircuit 구조체

파일: `zkevm-circuits/src/mpt_circuit.rs` (line 35-41)

```
pub struct MptCircuit<F: Field> {
row_limit: usize, // 최대 행 수
proofs: Vec<Proof>, // Merkle 증명 목록
mpt_updates: witness::MptUpdates, // MPT 업데이트 목록
_phantom: std::marker::PhantomData<F>,
}
```

MptCircuitConfig 구조체

파일: `zkevm-circuits/src/mpt_circuit.rs` (line 54-59)

```
pub struct MptCircuitConfig<F: Field>(
pub(crate) mpt::MptCircuitConfig, // 외부 mpt-circuits의 설정
pub(crate) MptTable, // MPT 테이블
std::marker::PhantomData<F>,
);
```

회로 설정

파일: `zkevm-circuits/src/mpt_circuit.rs` (line 61-80)

```
impl SubCircuitConfig<Fr> for MptCircuitConfig<Fr> {
type ConfigArgs = MptCircuitConfigArgs;

fn new(
```

```
meta: &mut ConstraintSystem<Fr>,

Self::ConfigArgs {

poseidon_table,

mpt_table,

challenges,

}: Self::ConfigArgs,

) -> Self {

// mpt-circuits의 설정 생성

let  conf = mpt::MptCircuitConfig::configure(

meta,

challenges.evm_word(),

&poseidon_table

);

// MPT Table lookup 설정

meta.lookup_any("updates in mpt table proven in mpt circuit", |meta| {

mpt_table

.table_exprs(meta)

.into_iter()

.zip_eq(conf.lookup_exprs(meta))

.collect()

});



Self(conf, mpt_table, Default::default())
```

```
}

}
```

Witness 생성

파일: `zkevm-circuits/src/mpt_circuit.rs` (line 82-110)

```
impl SubCircuit<Fr> for MptCircuit<Fr> {

fn  new_from_block(block: &witness::Block) -> Self {

// 최소 행 수 검증

if  block.circuits_params.max_mpt_rows != 0 {

if  block.circuits_params.max_mpt_rows < 2049 {

panic!("invalid max_mpt_rows {}", block.circuits_params.max_mpt_rows);

}

}

// SMTTrace를 Proof로 변환

let  traces: Vec<_> = block

.mpt_updates

.proof_types

.iter()

.cloned()

.zip_eq(block.mpt_updates.smt_traces.iter().cloned())

.collect();
```

```
Self {
proofs: traces.into_iter().map(Proof::from).collect(),
row_limit: block.circuits_params.max_mpt_rows,
mpt_updates: block.mpt_updates.clone(),
..Default::default()
}
}
}
```

최소 행 수 계산

파일: `zkevm-circuits/src/mpt_circuit.rs` (line 112-122)

```
fn min_num_rows_block(block: &witness::Block) -> (usize, usize) {
(
// 각 증명은 최대 3개의 키 조회 필요 (각 32행)
3 * 32 * block.mpt_updates.len(),
// 키 비트 조회는 최소 8 * 256 행 필요
block.circuits_params.max_mpt_rows.max(8 * 256 + 1),
)
}
```

설명:

- 빈 스토리지 증명도 최대 3개의 키 조회 필요

- 각 키 조회는 32행 필요
- 키 비트 조회는 8 * 256 = 2048행 필요 (+1은 오프셋)

회로 할당

파일: `zkevm-circuits/src/mpt_circuit.rs` (line 124-145)

```
fn synthesize_sub(
&self,
config: &Self::Config,
challenges: &Challenges<Value<Fr>>,
layouter: &mut impl Layouter<Fr>,
) -> Result<(), Error> {
// mpt-circuits의 증명 할당
config.0.assign(layouter, &self.proofs, self.row_limit)?;
// MPT Table 할당 (병렬 또는 순차)
let use_seq = std::env::var("PARALLEL_SYN").map_or(false, |s| s == *"false");
if !use_seq {
// 병렬 할당 (기본)
config.1.load_par(
layouter,
&self.mpt_updates,
self.row_limit,
challenges.evm_word(),
)?;
```

```
} else {
// 순차 할당
config.1.load(
layouter,
&self.mpt_updates,
self.row_limit,
challenges.evm_word(),
)?;
}
Ok(())
}
```

병렬 할당:

- 기본적으로 병렬 할당 사용 (`load_par`)
- `PARALLEL_SYN=false` 환경 변수로 순차 할당 가능
- 병렬 할당은 성능 향상

Poseidon Table 할당

파일: `zkevm-circuits/src/mpt_circuit.rs` (line 170-180)

```
fn synthesize(
&self,
(mpt_config, poseidon_table, challenges): Self::Config,
mut layouter: impl Layouter<Fr>,
```

```
) -> Result<(), Error> {
// Poseidon 해시 트레이스 생성
let poseidon_table_rows: Vec<_> = hash_traces(&self.proofs)
.iter()
.map(|([left, right], domain, hash)| {
[*hash, *left, *right, Fr::zero(), *domain, Fr::one()].map(Value::known)
})
.collect();
// Poseidon Table 할당
poseidon_table.load(&mut layouter, &poseidon_table_rows)?;
// MPT Circuit 할당
let challenges = challenges.values(&layouter);
self.synthesize_sub(&mpt_config, &challenges, &mut layouter)
}
```

WitnessGenerator (증인 생성기)

구조체

파일: `zkevm-circuits/src/witness/mpt/witness.rs` (line 40-45)

```
pub struct WitnessGenerator {
trie: ZkTrie, // 계정 트리
storages_cache: HashMap<Address, ZkTrie>, // 스토리지 트리 캐시
```

```
ref_db: Rc<ZkMemoryDb>, // 공유 데이터베이스
}
```

주요 메서드

1. handle_new_state (상태 업데이트 처리)

파일: `zkevm-circuits/src/witness/mpt/witness.rs` (line 360-380)

```
pub fn handle_new_state(
&mut self,
proof_type: MPTProofType,
address: Address,
new_val: Word,
old_val: Word,
key: Option<Word>,
) -> SMTTrace {
if let Some(key) = key {
// 스토리지 업데이트
self.trace_storage_update(address, key, new_val, old_val)
} else {
// 계정 업데이트
self.trace_account_update(address, |acc_before| {
// proof_type에 따라 계정 필드 업데이트
// ...
```

```
})
}
}
```

2. trace_account_update (계정 업데이트 추적)

파일: `zkevm-circuits/src/witness/mpt/witness.rs` (line 250-320)

```
fn trace_account_update<U>(&mut self, address: Address, update_account_data: U
where
U: FnOnce(Option<&AccountData>) -> Option<AccountData>,
{
// 1. 업데이트 전 증명 생성
let proofs = self.trie.prove(address.as_bytes()).unwrap();
let address_key = hash_zktrie_key(&extend_address_to_h256(&address));
let (account_path_before, account_data_before) =
decode_proof_for_mpt_path(address_key, proofs).unwrap();
// 2. 계정 데이터 업데이트
let account_data_after = update_account_data(account_data_before.as_ref());
// 3. 트리 업데이트
if let Some(account_data_after) = account_data_after {
self.trie.update_account(address.as_bytes(), &acc_data).unwrap();
} else if account_data_before.is_some() {
self.trie.delete(address.as_bytes());
```

```
}

// 4. 업데이트 후 증명 생성

let  proofs = self.trie.prove(address.as_bytes()).unwrap();

let (account_path_after, _) = decode_proof_for_mpt_path(address_key, proofs).unw

// 5. SMTTrace 생성

SMTTrace {

address: HexBytes(address.0),

account_path: [account_path_before, account_path_after],

account_update: [account_data_before.map(to_smt_account),

account_data_after.map(to_smt_account)],

account_key: smt_hash_from_u256(&address_key),

// ...

}

}
```

3. trace_storage_update (스토리지 업데이트 추적)

파일: `zkevm-circuits/src/witness/mpt/witness.rs` (line 120-220)

```
fn  trace_storage_update(

&mut  self,

address: Address,

key: Word,

new_value: Word,
```

```
old_value: Word,
) -> SMTTrace {
// 1. 스토리지 키 해시
let (storage_key, key) = {
let mut word_buf = [0u8; 32];
key.to_big_endian(word_buf.as_mut_slice());
(hash_zktrie_key(&word_buf), HexBytes(word_buf))
};
// 2. 스토리지 트리 가져오기
let trie = self.fetch_storage_cache(address).unwrap();
// 3. 업데이트 전 증명
let storage_before_proofs = trie.prove(key.as_ref()).unwrap();
let storage_before = decode_proof_for_mpt_path(storage_key, storage_before_proo
// 4. 스토리지 트리 업데이트
if !new_value.is_zero() {
trie.update_store(key.as_ref(), &new_value).unwrap();
} else if !old_value.is_zero() {
trie.delete(key.as_ref());
}
// 5. 업데이트 후 증명
let storage_root_after = H256(trie.root());
let storage_after_proofs = trie.prove(key.as_ref()).unwrap();
let storage_after = decode_proof_for_mpt_path(storage_key, storage_after_proofs
```

```
// 6. 계정 업데이트 (storage_root 변경)
let mut out = self.trace_account_update(address, |acc| {
if let Some(acc) = acc {
let mut acc = *acc;
acc.storage_root = storage_root_after;
Some(acc)
} else {
None
}
});
// 7. 스토리지 정보 추가
out.common_state_root = None;
out.state_key = Some(smt_hash_from_u256(&storage_key));
out.state_path = [storage_before.map(|(p, _)| p).ok(),
storage_after.map(|(p, _)| p).ok()];
out.state_update = Some([Some(store_before), Some(store_after)]);
out
}
```

키 해싱 (Secure Hash)

파일: `zkevm-circuits/src/witness/mpt/witness.rs` (line 500-515)

```
fn hash_zktrie_key(key_buf: &[u8; 32]) -> Word {
```

```rust
use halo2_proofs::halo2curves::bn256::Fr;
use hash_circuit::hash::Hashable;

// 키를 두 개의 128비트 값으로 분할
let first_16bytes: [u8; 16] = key_buf[..16].try_into().unwrap();
let last_16bytes: [u8; 16] = key_buf[16..].try_into().unwrap();

let bt_high = Fr::from_u128(u128::from_be_bytes(first_16bytes));
let bt_low = Fr::from_u128(u128::from_be_bytes(last_16bytes));

// Poseidon 해시 (domain = 512)
let hash = Fr::hash_with_domain([bt_high, bt_low], Fr::from(SECURE_HASH_DOMAIN)

U256::from_little_endian(hash.to_repr().as_ref())
}
```

SECURE_HASH_DOMAIN:

- 값: 512
- 용도: Poseidon 해시의 도메인 분리
- 파일: `zktrie/src/state/builder.rs` (line 36)

증명 디코딩

파일: `zkevm-circuits/src/witness/mpt/witness.rs` (line 540-620)

```
fn decode_proof_for_mpt_path(
mut key: Word,
proofs: Vec<Vec<u8>>,
) -> Result<(SMTPath, Option<ZkTrieNode>), IoError> {
// 1. 루트 해시 추출
let root = if let Some(arr) = proofs.first() {
let n = ZkTrieNode::parse(arr.as_slice()).unwrap();
smt_hash_from_bytes(n.node_hash().as_slice())
} else {
HexBytes::<32>([0; 32])
};
// 2. 증명을 TrieProof로 변환
let proof_bytes = proofs.iter().map(Vec::as_slice);
let trie_proof = TrieProof::<LeafNode>::try_from(BytesArray(proof_bytes))?;
// 3. 경로 구축
let mut path_bit_now = BigUint::from(1_u32);
let mut path_part: BigUint = Default::default();
let mut path = Vec::new();


for ((left, right), &node_type) in trie_proof.path.iter().zip(&trie_proof.path_
let is_bit_one = key.bit(0); // 최하위 비트 확인
path.push(if is_bit_one {
```

```
SMTNode {
node_type,
value: smt_hash_from_u256(right), // 오른쪽 자식
sibling: smt_hash_from_u256(left), // 왼쪽 형제
}
} else {
SMTNode {
node_type,
value: smt_hash_from_u256(left), // 왼쪽 자식
sibling: smt_hash_from_u256(right), // 오른쪽 형제
}
});
key >>= 1; // 다음 비트로 이동
if  is_bit_one {
path_part += &path_bit_now
};
path_bit_now *= 2_u32;
}
// 4. 리프 노드 처리
let  node = trie_proof.data.0;
let  leaf = trie_proof.key.as_ref().map(|h| SMTNode {
node_type: trie_proof.key_type.unwrap(),
value: smt_hash_from_bytes(&node.as_ref().unwrap().value_hash().unwrap()),
```

```
sibling: smt_hash_from_bytes(h.as_bytes()),
});
Ok((SMTPath { root, leaf, path, path_part }, node))
}
```

Poseidon Table 연동

PoseidonLookup 트레이트 구현

파일: `zkevm-circuits/src/mpt_circuit.rs` (line 20-30)

```
impl PoseidonLookup for PoseidonTable {
fn  lookup_columns_generic(&self) -> (Column<Fixed>, [Column<Advice>; 6]) {
(
self.q_enable,
[
self.hash_id,
self.input0,
self.input1,
self.control,
self.domain_spec,
self.heading_mark,
],
)
```

```
}

}
```

Poseidon 해시 트레이스

MPT Circuit은 모든 Poseidon 해시 연산을 추적하여 Poseidon Table에 할당합니다:

```
// 해시 트레이스 생성

let  poseidon_table_rows: Vec<_> = hash_traces(&self.proofs)

.iter()

.map(|([left, right], domain, hash)| {

[

*hash, // 해시 결과

*left, // 왼쪽 입력

*right, // 오른쪽 입력

Fr::zero(), // control (사용 안 함)

*domain, // 도메인 (512)

Fr::one(), // heading_mark (활성화)

].map(Value::known)

})

.collect();
```

Poseidon 해시 사용

MPT Circuit에서 Poseidon 해시는 다음 용도로 사용됩니다:

1. **키 해싱**: 계정 주소와 스토리지 키를 해시
2. **노드 해싱**: 트리 노드의 해시 계산
3. **루트 계산**: 트리 루트 해시 계산

예시:

```
// 계정 주소 해싱
let address_key = hash_zktrie_key(&extend_address_to_h256(&address));


// 노드 해싱 (left, right 자식)
let node_hash = poseidon_hash([left_child_hash, right_child_hash], domain);
```

전체 업데이트 흐름

1. 상태 변경 수집

Bus-Mapping 단계에서 모든 상태 변경을 수집합니다:

```
// 예시: 계정 잔액 변경
RwOp::Account {
address: 0x1234,
field_tag: AccountFieldTag::Balance,
value: 100,
value_prev: 50,
}
```

2. MPT 업데이트 생성

각 상태 변경에 대해 `WitnessGenerator` 를 사용하여 MPT 업데이트를 생성합니다:

```
let mut witness_gen = WitnessGenerator::from(&state);
```

```
let smt_trace = witness_gen.handle_new_state(
MPTProofType::BalanceChanged,
address,
new_value, // 100
old_value, // 50
None, // 계정 업데이트 (스토리지 키 없음)
);
```

3. SMTTrace 생성

`handle_new_state` 는 다음을 수행합니다:

1. **업데이트 전 증명 생성**: 현재 트리 상태에서 증명 생성
2. **트리 업데이트**: zkTrie에 새 값 적용
3. **업데이트 후 증명 생성**: 업데이트된 트리에서 증명 생성
4. **SMTTrace 구축**: 두 증명을 결합하여 SMTTrace 생성

4. Proof 변환

`SMTTrace` 를 `Proof` 로 변환합니다:

```
let proof = Proof::from((proof_type, smt_trace));
```

5. 회로 할당

MPT Circuit에 증명을 할당합니다:

```
// mpt-circuits의 증명 할당
config.0.assign(layouter, &self.proofs, self.row_limit)?;


// MPT Table 할당
config.1.load_par(layouter, &self.mpt_updates, self.row_limit, challenges.evm_wo
```

6. Poseidon Table 할당

모든 Poseidon 해시 연산을 Poseidon Table에 할당합니다:

```
let  poseidon_table_rows = hash_traces(&self.proofs);
poseidon_table.load(&mut  layouter, &poseidon_table_rows)?;
```

예시: 전체 트랜잭션 처리

Solidity 코드

```
contract Example {
uint256 public value;
function setValue(uint256 newValue) public {
```

```
value = newValue;
}
}
```

트랜잭션 실행

```
Sender: 0xABCD...
Contract: 0x1234...
Function: setValue(42)
Gas Price: 10 gwei
Gas Used: 50000
```

상태 변경

1. **Sender Nonce 증가**: 5 → 6
2. **Sender Balance 감소**: 100 ETH → 99.5 ETH (가스 지불)
3. **Contract Storage 변경**: value = 0 → 42

MPT 업데이트 시퀀스

업데이트 1: Sender Nonce

```
witness_gen.handle_new_state(
MPTProofType::NonceChanged,
sender_address,
```

```
Word::from(6),
Word::from(5),
None,
);
```

SMTTrace:

```
account_path[0]: Root1 → ... → Sender (nonce=5)
account_path[1]: Root2 → ... → Sender (nonce=6)
account_update: [Some(Account{nonce:5, ...}), Some(Account{nonce:6, ...})]
```

업데이트 2: Sender Balance

```
witness_gen.handle_new_state(
MPTProofType::BalanceChanged,
sender_address,
Word::from(99_500_000_000_000_000_000u128), // 99.5 ETH
Word::from(100_000_000_000_000_000_000u128), // 100 ETH
None,
);
```

SMTTrace:

```
account_path[0]: Root2 → ... → Sender (balance=100 ETH)
account_path[1]: Root3 → ... → Sender (balance=99.5 ETH)
```

```
account_update: [Some(Account{balance:100, ...}), Some(Account{balance:99.5, ...
```

업데이트 3: Contract Storage

```
witness_gen.handle_new_state(
MPTProofType::StorageChanged,
contract_address,
Word::from(42),
Word::from(0),
Some(Word::from(0)), // storage slot 0
);
```

SMTTrace:

```
account_path[0]: Root3 → ... → Contract (storage_root=SR1)
account_path[1]: Root4 → ... → Contract (storage_root=SR2)
state_path[0]: SR1 → ... → Empty
state_path[1]: SR2 → ... → Leaf (value=42)
state_update: [Some(StateData{value:0}), Some(StateData{value:42})]
```

루트 연속성

각 업데이트의 `new_root` 가 다음 업데이트의 `old_root` 와 일치해야 합니다:

```
Root1 (초기 상태)
```

```
↓ (Nonce 변경)

Root2

↓ (Balance 변경)

Root3

↓ (Storage 변경)

Root4 (최종 상태)
```

MPT Table

```
address | storage_key | proof_type | new_root | old_root | new_value | old_value

---------|-------------|-----------------|----------|----------|-----------|----

0xABCD.. | 0 | NonceChanged | Root2 | Root1 | 6 | 5

0xABCD.. | 0 | BalanceChanged | Root3 | Root2 | 99.5 ETH | 100 ETH

0x1234.. | key_rlc | StorageChanged | Root4 | Root3 | 42 | 0
```

성능 최적화

1. Poseidon 해시

장점:

- SNARK-friendly: 회로 크기 대폭 감소
- Keccak256 대비 10배 이상 효율적
- 제약 조건 수 감소

비교:

```
Keccak256: ~150,000 제약 조건

Poseidon: ~15,000 제약 조건
```

2. Sparse Binary Tree

장점:

- 메모리 효율적: 대부분의 노드가 비어있음
- 증명 크기 감소: Binary tree는 Hexary tree보다 작음
- 경로 압축: 긴 경로를 압축하여 저장

비교:

```
Hexary MPT: 평균 증명 크기 ~10KB

zkTrie: 평균 증명 크기 ~5KB
```

3. 병렬 할당

파일: `zkevm-circuits/src/mpt_circuit.rs` (line 135-145)

```
// 병렬 할당 (기본)
config.1.load_par(layouter, &self.mpt_updates, self.row_limit, challenges.evm_wo


// 순차 할당 (PARALLEL_SYN=false)
config.1.load(layouter, &self.mpt_updates, self.row_limit, challenges.evm_word()
```

성능 향상:

- 병렬 할당은 멀티코어 CPU 활용
- 대규모 블록에서 2-3배 성능 향상

4. 캐싱

파일: `zkevm-circuits/src/witness/mpt/witness.rs` (line 100-115)

```
fn fetch_storage_cache(&mut self, address: Address) -> Option<&mut ZkTrie> {
let cache_entry = self.storages_cache.entry(address);
match cache_entry {
std::collections::hash_map::Entry::Vacant(entry) => {
let account = self.trie.get_account(address.as_bytes());
account
.map(AccountData::from)
.and_then(|acc_data| {
self.ref_db.new_trie(&acc_data.storage_root.0)
})
.map(|trie| entry.insert(trie))
}
std::collections::hash_map::Entry::Occupied(entry) => Some(entry.into_mut()),
}
}
```

장점:

- 스토리지 트리를 캐싱하여 반복 조회 방지

- 동일 계정의 여러 스토리지 업데이트 시 성능 향상

제약 조건

1. 루트 연속성

```
// 연속된 업데이트의 루트가 연결되어야 함
for i in 0..updates.len() - 1 {
assert_eq!(updates[i].new_root, updates[i+1].old_root);
}
```

2. Merkle 증명 유효성

```
// 각 업데이트에 대한 유효한 Merkle 증명
verify_proof(old_root, key, old_value, path_before) == true
verify_proof(new_root, key, new_value, path_after) == true
```

3. 값 일관성

```
// RW Table의 값과 MPT Table의 값이 일치
rw_table.value == mpt_table.new_value
rw_table.value_prev == mpt_table.old_value
```

4. 키 일치

```
// 증명의 키가 예상 키와 일치
proof.key == expected_key
```

5. 노드 타입 유효성

```
// 노드 타입이 유효한 범위 (4-9)
assert!(node_type >= 4 && node_type <= 9);
```

디버깅 팁

MPT 업데이트 덤프

파일: `zkevm-circuits/src/witness/mpt/witness.rs` (line 70-78)

```
pub fn dump<'a>(&self, addrs: impl Iterator<Item = &'a Address>) {
for addr in addrs {
let acc = self
.trie
.get_account(addr.as_bytes())
.map(AccountData::from);
log::info!("account data {:#x}: {:#?}", addr, acc);
}
}
```

루트 연속성 검증

```
// 루트 연속성 확인
for i in 0..updates.len() - 1 {
if updates[i].new_root != updates[i+1].old_root {
log::error!("Root mismatch at index {}: {:?} != {:?}",
i, updates[i].new_root, updates[i+1].old_root);
}
}
```

증명 검증

```
// 증명 유효성 확인
let proof = trie.prove(address.as_bytes()).unwrap();
let (path, node) = decode_proof_for_mpt_path(address_key, proof).unwrap();

if node.is_none() {
log::warn!("Proof verification failed for address {:?}", address);
}
```

일반적인 오류

1. 루트 불일치

증상: `Root mismatch at index X`

원인:

- 업데이트 순서가 잘못됨
- 중간 업데이트가 누락됨
- 트리 업데이트가 제대로 적용되지 않음

해결:

```
// 업데이트 순서 확인
for (i, update) in updates.iter().enumerate() {
log::info!("Update {}: {:?} -> {:?}", i, update.old_root, update.new_root);
}
```

2. 증명 실패

증상: `Proof verification failed`

원인:

- 키가 잘못 해시됨
- 트리 데이터베이스가 불완전함
- 노드 데이터가 손상됨

해결:

```
// 키 해싱 확인
let address_key = hash_zktrie_key(&extend_address_to_h256(&address));
log::info!("Address: {:?}, Key: {:?}", address, address_key);

// 트리 데이터베이스 확인
```

```
let  node = trie.get_account(address.as_bytes());

log::info!("Node data: {:?}", node);
```

3. 값 불일치

증상: `Value mismatch: expected X, got Y`

원인:

- RW Table과 MPT Table의 값이 다름
- 업데이트 전후 값이 잘못됨

해결:

```
// 값 일관성 확인

assert_eq!(rw_op.value, mpt_update.new_value,

"Value mismatch for address {:?}", address);

assert_eq!(rw_op.value_prev, mpt_update.old_value,

"Old value mismatch for address {:?}", address);
```

4. 스토리지 트리 오류

증상: `Storage trie not found for address X`

원인:

- 계정이 존재하지 않음
- 스토리지 루트가 잘못됨

해결:

```
// 계정 존재 확인
let account = trie.get_account(address.as_bytes());
if account.is_none() {
log::error!("Account does not exist: {:?}", address);
}

// 스토리지 루트 확인
let acc_data = AccountData::from(account.unwrap());
log::info!("Storage root: {:?}", acc_data.storage_root);
```

테스트

단위 테스트

파일: `zkevm-circuits/src/witness/mpt/witness.rs` (line 625-780)

1. witgen_init_writer

```
#[test]
fn witgen_init_writer() {
let (state, _, _) = build_state_from_string(EXAMPLE_TRACE);
let w = WitnessGenerator::from(&state);
let root_init = w.root();
assert_eq!(
```

```
format!("{root_init:?}"),
"0x24c368802ea77a0d8d49d8ccce69cdb7aead98533c77aebbd7605358a592f3aa"
);
}
```

목적: WitnessGenerator 초기화 및 루트 해시 확인

2. witgen_update_one

```
#[test]
fn witgen_update_one() {
let (state, accounts, storages) = build_state_from_string(EXAMPLE_TRACE);
let mut w = WitnessGenerator::from(&state);
// 잔액 변경
let trace = w.handle_new_state(
MPTProofType::BalanceChanged,
target_addr,
start_state.balance + U256::from(1_u64),
start_state.balance,
None,
);
let new_root = w.root();
let new_acc_root = smt_bytes_to_hash(trace.account_path[1].root.as_ref());
assert_eq!(new_root.0, new_acc_root);
```

```
}
```

목적: 단일 업데이트 및 루트 일관성 확인

통합 테스트

MPT Circuit은 다른 회로들과 함께 SuperCircuit에서 테스트됩니다:

```rust
#[test]
fn test_mpt_circuit_integration() {
// 1. 블록 생성
let block = generate_test_block();
// 2. MPT Circuit 생성
let mpt_circuit = MptCircuit::new_from_block(&block);
// 3. 증명 생성
let prover = MockProver::run(k, &mpt_circuit, vec![]).unwrap();
// 4. 검증
prover.assert_satisfied();
}
```

관련 문서

- State Circuit - MPT Table을 조회하는 회로
- MPT Table - MPT 테이블 상세
- Poseidon Circuit - Poseidon 해시 회로

- RW Table - 상태 변경 추적
- Bus-Mapping - 상태 변경 수집

참고 자료

코드

- **MPT Circuit**: `zkevm-circuits/src/mpt_circuit.rs`
- **Witness Generator**: `zkevm-circuits/src/witness/mpt/witness.rs`
- **zkTrie**: `zktrie/src/`
- **mpt-circuits**: 외부 크레이트 (Scroll의 MPT 회로 구현)

문서

- **Poseidon Hash**: https://www.poseidon-hash.info/
- **zkTrie Specification**: https://github.com/scroll-tech/zktrie-ng
- **이더리움 Yellow Paper**: https://ethereum.github.io/yellowpaper/paper.pdf (Appendix D - Modified Merkle Patricia Trie)
- **Scroll Documentation**: https://docs.scroll.io/

논문

- **Poseidon**: “Poseidon: A New Hash Function for Zero-Knowledge Proof Systems” (2019)
- **zkSNARK**: “Succinct Non-Interactive Zero Knowledge for a von Neumann Architecture” (2013)